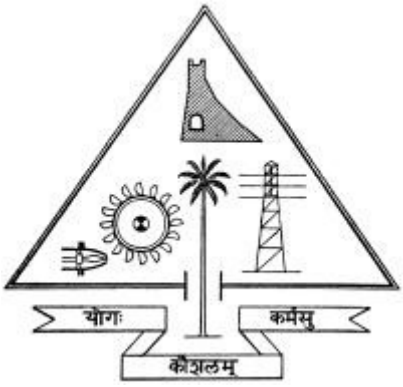
AI-POWERED CODE DOCUMENT GENERATOR

Main Project Report submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications of the APJ Abdul Kalam Technological University

submitted by

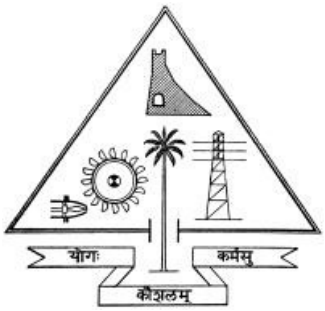
ARYA K A
(TCR24MCA-2020)



DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE
THRISSUR - 680009

MARCH 2026

DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE, THRISSUR
THRISSUR, KERALA STATE, PIN 680009



CERTIFICATE

This is to certify that the main project titled "AI-POWERED CODE DOCUMENT GENERATOR" is a bonafide work done by ARYA K A (TCR24MCA-2020) under my supervision and guidance, and is submitted in March 2026 in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications from APJ Abdul Kalam Technological University (KTU).

Prof. Sreejith V P	Prof. Raseek C	Dr. Sminesh C N
Project Guide	Project Coordinator	HOD

Place : Thrissur
Date : 31-03-2026

DECLARATION

I hereby declare that the main project named, AI-POWERED CODE DOCUMENT GENERATOR, is my own work and that, to the best of my knowledge and belief, it contains no material previously published by another person nor material which has been accepted for the award of any other degree or course of the university or any other institute of higher learning, except where due acknowledgment and reference has been made in the text.

Place : Thrissur
Date : 31-03-2026

ARYA K A
(TCR24MCA-2020)

ACKNOWLEDGEMENT

I would like to thank Department of Computer Application, GEC Thrissur, for giving me this opportunity to pursue this project and successfully complete it.

I am highly indebted to our head of the department Dr. Sminesh C N, for his guidance and constant supervision as well as providing necessary information regarding the project and also for his support in completing the project.

I express my heartfelt gratitude to my project guide Prof. Sreejith V P, for his committed guidance, valuable suggestions, constructive criticisms and precious time that he invested throughout the work.

I express my heartfelt gratitude to project coordinator Prof. Raseek C, for his committed guidance as well as providing necessary information regarding the project, and also for his support in completing the project.

I sincerely thank all other faculties of MCA department for guiding me through the processes involved in the project.

ABSTRACT

Maintaining software project documentation accurately and up to date is hard because code changes often, and writing documentation by hand takes a lot of time. Outdated documentation can make code harder to understand, require more effort to keep up to date, and pose challenges for new developers who want to join a project. To solve these problems, this project shows off an AI-Powered Code Document Generator that looks at the source code of a software project and automatically creates structured documentation.

The system can read code written in different programming languages from local directories or optional version-controlled repositories. It uses Abstract Syntax Tree (AST) parsing to get information about the structure like functions, classes, and dependencies, which help you understand the codebase better. After that, these extracted features are given to a Large Language Model (LLM), which turns the technical details into clear, easy-to-read documentation while keeping the accuracy and context.

The system makes a lot of different documents, such as a README file, architectural descriptions, file-wise explanations, and a list of recent changes. The proposed method makes it easier for developers to maintain software by automating the documentation process without changing the original source code. In general, this project shows how well artificial intelligence can be used to make technical documentation that is accurate, well-organized, and easy to understand.

CONTENTS

List of Figures vii

List of Abbreviations viii

1 INTRODUCTION 1

2 LITERATURE REVIEW 4

3 ENVIRONMENTAL STUDY 10
3.1 System Configuration . . . 10
3.1.1 Hardware Requirements . . . 11
3.1.2 Software Requirements . . . 11
3.1.3 Software Requirements for Development . . . 12
3.2 Software Specification . . . 12
3.2.1 Python 3.8 . . . 13
3.2.2 Visual Studio Code . . . 13
3.2.3 Tree-sitter . . . 14
3.2.4 Large Language Model(LLM) . . . 14
3.2.5 Git . . . 15
3.2.6 NetworkX . . . 15
3.2.7 Markdown . . . 15

4 SYSTEM ANALYSIS 17
4.1 Requirements Analysis . . . 17
4.2 Existing System . . . 18
4.3 Limitations of Existing System . . . 18
4.4 Proposed System . . . 19
4.5 Advantages of Proposed System . . . 19
4.6 Functional Requirements . . . 20
4.6.1 User Interface . . . 20
4.6.2 Source Code Input . . . 21
4.6.3 Source Code Parsing . . . 21
4.6.4 Feature Extraction . . . 21
4.6.5 LLM-Based Documentation Generation . . . 21
4.6.6 Documentation Output . . . 22
4.7 Non-Functional Requirements . . . 22
4.7.1 Scalability . . . 23
4.7.2 Usability . . . 23
4.7.3 Reliability . . . 23
4.7.4 Maintainability . . . 23
4.8 Feasibility Study . . . 24
4.8.1 Technical Feasibility . . . 24
4.8.2 Operational Feasibility . . . 24
4.8.3 Economical Feasibility . . . 24

5 SYSTEM DESIGN 26

5.1 System Architecture 26
5.2 Data Flow Diagram (DFD) 27
5.2.1 DFD Level 0 27
5.2.2 DFD Level 1 28
5.3 List of Modules 29
5.3.1 User Interface Module 30
5.3.2 Language Detection Module 30
5.3.3 Text Feature Extraction Module 30
5.3.4 AST Analysis Module 30
5.3.5 Graph Building Module 31
5.3.6 Feature Fusion Module 31
5.3.7 LLM Engine Module 31
5.3.8 Documentation Output Module 31

6 SYSTEM IMPLEMENTATION 33
6.1 Setup Development environment 33
6.2 User Interface 34
6.3 Language Detection 35
6.4 Text Feature Extraction 36
6.5 AST Analysis 36
6.6 Graph Analysis 37
6.7 Feature Fusion 38
6.8 Documentation Generation 39

7 RESULTS AND DISCUSSION 41
7.1 Results 41
7.2 Discussion 45

8 CONCLUSION 46

9 FUTURE ENHANCEMENT 48
9.1 Git Integration 48
9.2 Multi-Language Expansion 48
9.3 Interactive Q&A 48

Bibliography 51

LIST OF FIGURES

5.1 Architecture Diagram of the AI-Powered Code Documentation Generator. 27
5.2 Data Flow Diagram Level 0. 28
5.3 Data Flow Diagram Level 1. 29

7.1 Complete User Interface of the System. 42
7.2 Home Page – Repository URL Input Mode. 42
7.3 README.md – Project Overview Output. 43
7.4 architecture_overview.md – Architectural Description Output. . 43
7.5 File-wise .md Output. 44
7.6 recent_changes.md – Git Commit History Output. 45

LIST OF ABBREVIATIONS

AI	–	Artificial Intelligence
API	–	Application Programming Interface
AST	–	Abstract Syntax Tree
BART	–	Bidirectional and Auto-Regressive Transformers
BLEU	–	Bilingual Evaluation Understudy
CD	–	Continous Integration
CI	–	Continous Deployment
CSS	–	Cascading Style Sheets
DFD	–	Data Flow Diagram
GPT	–	Generative Pre-trained Transformer
HTML	–	HyperText Markup Language
IDE	–	Integrated Development Environment
LLM	–	Large Language Model
METEOR	–	Metric for Evaluation of Translation with Explicit Ordering
Q/A	–	Question and Answer
REST API	–	Representational State Transfer Application Programming Interface
RoBERTa	–	Robustly Optimized BERT Approach
ROUGE	–	Recall-Oriented Understudy for Gisting Evaluation
T5	–	Text-to-Text Transfer Transformer
URL	–	Uniform Resource Locator
UTF-8	–	8-bit Unicode Transformation Format
VS Code	–	Visual Studio Code

CHAPTER 1
INTRODUCTION

The rise in software system's development and the rising complexity in application development processes has made the need for good code documentation necessary. Software systems tend to become more advanced in terms of features, connections and development teams. Code documentation helps make code readable, encourages teamwork between programmers and guarantees software system maintainability in the future. Documentation acts as a link between programmers by offering an explanation of system features, design choices and coding processes. In large projects involving many programmers working on the project for a long time, documentation is vital to ensuring consistency and minimizing misunderstandings. Nonetheless, keeping documentation up to date and accurate is difficult because there are many modifications to code and it is time-consuming to write documentation manually.

Often, in practice, in many software development projects, it is observed that documentation tends to fall behind and there are chances that the documentation might not be complete for the software developed. In such situations, the developers do not have enough time to document the changes made to the codebase, owing to their focus being diverted to implementing features and fixing bugs. It also results in an inconsistency in the actual state of the codebase and what is described in the documentations related to the code. This makes it challenging for developers to understand the behavior of the system and hence spend extra time debugging and analyzing the actual code.

For addressing these difficulties, a set of latest development of Artificial Intelligence (AI) technologies can be used to automate documentation creation. The analytical technique of Abstract Syntax Tree (AST) is one of the most useful ways of analyzing source codes by building an abstract syntactic tree. Such components include functions, classes, variables and control statements, thus helping to have a clear understanding of the structure of source codes. Compared to traditional text approaches, analysis through AST allows to get more precise results and insights on behavior of the source codes analyzed.

Apart from the AST-based analysis, another way of gaining deeper insight into code by analyzing relationships among its different elements is by means of graphical representation. In this method, nodes are represented using objects such as functions, classes and modules, while edges are represented using relationships between them, such as function call dependencies and inheritance. Graphs can be very useful to identify relationships and patterns in the form of dependencies, control flow and the most important components in the system. So, together with the AST analysis method, graphs give a broader perspective on the code base.

In addition, Large Language Models (LLMs) have brought another evolution in the documentation process, where the complex and complicated structure of code can be translated into understandable and readable text. The detailed grasp of the semantics of the code by LLMs is useful in making fine descriptions, summaries and documentation of the code itself. Using structured input based on the analysis of an abstract syntax tree and graphs, LLMs can provide precise, consistent and contextually relevant documentation, README files and architectural documentation of modules and functions.

The goal of this project is to create an intelligent system for automatic generation of documentation based on source code. This will be achieved through the use of AST-based parsing and modeling techniques, as well as natural language models, to reduce human interference, enhance software readability and maintainability. This intelligent system will aid developers in

creating accurate and updated documentation without having to modify the source code.

The increasing demand for intelligent and scalable documentation highlights the limitations of conventional methods in handling complex software systems. Manual documentation processes are often slow, inconsistent and unable to keep up with rapid development cycles and frequent software updates. As a result, maintaining accurate and up-to-date documentation becomes a significant challenge.

In contrast, integrating structural analysis techniques with state-of-the-art language models offers a more effective solution by enabling automated and context-aware documentation generation. Therefore, reviewing the existing literature is essential to identify current approaches, their limitations and potential areas for improvement, which are further explored in the following chapter.

CHAPTER 2
LITERATURE REVIEW

Maintaining accurate and up-to-date software documentation remains one of the greatest challenges in contemporary software development. As software systems continue to evolve through continuous integration and agile methodologies, documentation created manually often becomes inconsistent with the source code. This mismatch between code and its documentation reduces code legibility and makes it challenging for new developers joining a codebase. To address these problems, researchers have recently made efforts toward automating document generation using artificial intelligence. In particular, combining structural program analysis with LLM has emerged as a promising solution. This chapter reviews the most relevant literature, organized around key themes including code representation strategies, repository-level documentation approaches, empirical evaluations of language models, and practical deployment considerations.

A fundamental challenge in automated code documentation is how to represent source code in a way that captures its meaning and structure for machine processing. Early approaches treated code as simple sequences of tokens, but this ignored the rich syntactic structure that distinguishes programming languages from natural language. Researchers have increasingly recognized that AST offer a more powerful representation, as they break code down into meaningful components such as functions, classes, and control statements while preserving hierarchical relationships. However, simply serializing ASTs into sequences for neural network processing loses important positional relationships between nodes, including hierarchical parent-child relations and sibling order. To address this limitation, researchers have

proposed tree positional embeddings that inject both vertical and horizontal positional information for each node in the AST, along with specialized syntax-tree attention mechanisms that focus on syntactically important code tokens. This enhanced AST encoding has demonstrated consistent improvements across multiple programming languages and evaluation metrics, outperforming state-of-the-art models by significant margins on both Java and Python benchmarks [1]. Beyond traditional AST representations, researchers have explored treating code as graphs to capture relationships that tree structures cannot adequately represent. When ASTs are transformed into graph representations where nodes correspond to syntactic units and edges represent hierarchical relationships, graph neural networks can extract structural features through neural message passing with neighborhood aggregation. This graph-based perspective preserves connectivity information that tree representations miss, such as complex dependency relationships and recursive patterns, and has achieved high accuracy in code understanding tasks [2]. Additionally, incorporating code metrics such as complexity measures, API (Application Programming Interface) popularity, and Halstead metrics into documentation models has been shown to improve generation quality, suggesting that structural characteristics beyond raw syntax contribute meaningfully to code comprehension [3].

The integration of multiple complementary code representations has emerged as another important direction for improving documentation quality. Rather than relying on a single representation, combining token sequences with type-augmented ASTs and control flow graphs provides a more complete picture of code semantics. Type augmentation enriches ASTs with type information about variables and expressions, while control flow graphs capture program execution order and branching structure. Hierarchical attention mechanisms have proven effective for integrating these diverse representations, with one attention layer operating at the token level within statements and another layer operating at the statement level within functions. This hierarchical structure mirrors the natural hierarchy of code,

where tokens form statements and statements form functions. Reinforcement learning has also been employed to address the exposure bias problem in sequence generation, where models trained to predict the next word given ground truth context must rely on their own potentially incorrect predictions during actual use. Actor-critic networks jointly optimize word selection and evaluate overall sequence quality, leading to substantial improvements over maximum-likelihood training approaches [4].

Documenting entire software repositories presents challenges beyond those of individual function summarization. Understanding a full software project requires comprehending inter-file dependencies, system architecture, and how different modules collaborate to implement overall functionality. One framework addresses this by constructing a project tree that reflects the repository's directory structure, then extracting reference relationships between code objects to form a directed graph that captures semantic call relationships. These reference relationships serve as natural contextual learning examples, teaching language models how different parts of the codebase interact. The framework generates structured documentation covering functionality, parameters, code descriptions, considerations, and examples, using prompt templates that incorporate project trees, code snippets, reference relationships, and metadata [5]. A feature-oriented approach to repository summarization offers an alternative to directory-based organization. Rather than relying solely on file structure, this method clusters methods and files by both dependency relationships and semantic similarity to identify functional features that naturally span multiple files. The clustering process uses community detection algorithms on graphs where nodes represent code elements and edges represent both structural dependencies and semantic similarities. Once features are identified, large language models generate descriptive documentation for each feature, and traceability links are preserved between features and the code elements that implement them. This feature-oriented organization has been shown to increase the rate of completely covered features in manual documentation and

significantly improve the recall of traceability links between documentation and code [6].

Beyond generating documentation directly from code, researchers have also explored the related task of generating code examples to complement existing documentation. This work recognizes that good documentation includes not only descriptions of what code does but also concrete examples of how to use it. Using language models pre-trained on both natural and programming languages, researchers have generated code examples from source code and documentation inputs. The effectiveness of this approach has been demonstrated through metrics such as passability, measuring whether generated examples execute without error, and relevance, measuring whether examples properly address the target method and documentation. A particularly significant finding is that incorporating compiler error logs into the input improves passability substantially, as the error messages provide feedback that allows the model to correct mistakes in subsequent attempts. This error-feedback mechanism demonstrates the value of iterative refinement based on execution results [7].

Empirical evaluation of different language models for documentation tasks has provided valuable guidance for model selection. Comparative studies have examined models such as GPT-2 (Generative Pre-trained Transformer–version 2) and RoBERTa (Robustly Optimized BERT Pretraining Approach), finding that RoBERTa consistently outperforms GPT-2 in both accuracy and loss metrics. The superior performance of RoBERTa is attributed to its ability to detect semantic connections in code, achieving near-perfect accuracy on documentation generation tasks while maintaining dramatically lower training and validation losses. These results highlight the importance of model architecture choices and the value of fine-tuning with domain-specific datasets for documentation generation [8]. Similar comparative work on T5 (Text-to-Text Transfer Transformer) and BART (Bidirectional and Auto-Regressive Transformer) has shown that it generates more concise and context-aware comments, making it more reliable for capturing the single

intent of code snippets. However, T5 offers advantages in training efficiency, presenting a trade-off between output quality and computational resources. The use of multiple evaluation metrics including BLEU (Bilingual Evaluation Understudy), ROUGE (Recall-Oriented Understudy for Gisting Evaluation), METEOR (Metric for Evaluation of Translation with Explicit ORdering), and Smoothed BLEU provides a comprehensive framework for assessing documentation quality across different dimensions [9].

The practical deployment of automated documentation systems in real-world development workflows has also received attention. One system integrates AST generation from a compiler with a large language model within a continuous integration and continuous delivery pipeline, triggered on every code push to automatically generate updated documentation. This integration with version control systems ensures that documentation remains synchronized with code changes, addressing the documentation debt problem directly. A survey of experienced developers found that concise documentation was strongly preferred over verbose output, with developers rating terseness as more important for comprehensibility than additional detail. This finding has important implications for prompt design and model configuration in automated documentation systems, suggesting that generating shorter, focused documentation may be more valuable than producing comprehensive but lengthy explanations [10].

In summary, the reviewed literature establishes that effective automated code documentation requires careful attention to code representation, with AST-based, graph-based, and metric-enriched approaches each contributing valuable perspectives. Repository-level documentation benefits from both directory-based and feature-oriented organization strategies. Empirical studies provide guidance on model selection, with different models offering trade-offs between output quality and computational efficiency. Practical deployment in CI/CD (Continous Integration/ Continous Deployment) pipelines has demonstrated feasibility, with user preferences favoring concise documentation. The proposed AI-Powered Code Document Generator

builds on these findings by combining Tree-sitter-based AST parsing across fourteen programming languages, NetworkX-based graph analysis for capturing function call relationships, and a locally-hosted LLM to generate comprehensive documentation including README files, architectural descriptions, file-wise explanations, and recent change histories without modifying source code.

Even though all the above-mentioned methods are required to create the proposed system, their implementation needs a certain environment with resources and tools prepared beforehand. That is why the proper setting up of the development environment will be important and this point is described in the next chapter.

CHAPTER 3
ENVIRONMENTAL STUDY

The environmental settings of the project are discussed in this chapter as it is essential in ensuring that the AI-Powered Code Document Generator system is successfully developed, implemented and assessed. Environment of the project is important because it helps system perform its functions effectively while ensuring that all its component are integrated.

Environment entails both hardware and software pre-requisites that may be required by the system to operate successfully under ideal scenarios. It emphasizes all the tools, frameworks and platforms used during system development. The configuration of these elements optimizes the performance of the system.

3.1 System Configuration

System configuration means the collection of hardware and software components necessary for developing and executing the system. The proper configuration helps to ensure that the system works efficiently and effectively. Source code is processed easily with the help of the right configuration. At the same time, it helps ensure good interaction between all modules within the system.

Operations that the system may perform include parsing, AST creation, feature extraction, and communication with LLM for documentation generation. Therefore, system configuration, based on an adequately developed environment, is essential for ensuring better overall system performance and less processing time during the testing and implementation phases so that, eventually, a system can run without glitches and lags.

3.1.1 Hardware Requirements

The hardware requirements outline the minimum resources necessary to operate the system. These resources are crucial for performing code analysis, processing tasks and running machine learning operations.

The key hardware requirements are detailed as follows:

- Processor: Intel® Core™ i5 Processor / AMD Ryzen™ 5 Processor or higher, to efficiently handle code processing tasks.
- Memory: Minimum 8 GB RAM, to ensure smooth execution of analysis and model operations.
- Storage: 50–100 GB of free disk space, for storing code files and generated outputs.

3.1.2 Software Requirements

The requirements for software specify the necessary platforms, languages and libraries that will be used in running the system. The use of these tools, activities like parsing, integration and documentation become easy. Compatibility assurance is relevant to ensure that the various components of the system communicate effectively.

The key software requirements are detailed as follows:

- Operating System: Windows 10, Ubuntu 20.04 LTS or macOS, to support cross-platform execution.
- Programming Language: Python 3.8 or higher, used for implementing the entire system.
- Version Control System: Git (optional), for managing repository-based inputs and commit history.
- Required Python Libraries:
  - Tree-sitter – a parser that converts source code into an AST, enabling precise analysis of functions, classes and other structures.

  - Markdown – a lightweight markup language for generating clear and easily readable documentation from code.
  - NetworkX – a Python library for creating and analyzing graphs, useful for visualizing relationships between functions, classes, and modules in source code.

3.1.3 Software Requirements for Development

Besides the execution specifications, some indispensable are the tools for development, testing and maintenance. They assist in writing and debugging programs, as well as managing the code appropriately. Besides, the tools help in maintaining versions and assembling different systems together.

The key development software requirements for development are detailed as follows:

- Operating System: Windows 10, Ubuntu 20.04 LTS or macOS, ensuring cross-platform compatibility.
- Back End: Python 3.8 or later, serving as the core implementation language.
- IDE Used: Visual Studio Code, used for development and debugging tasks.
- Version Control: Git, for managing source code and repositories.
- Development Tools and Frameworks:
  - Tree-sitter – for AST-based code analysis.
  - Large Language Model (Ollama DeepSeek) – for generating documentation efficiently.

3.2 Software Specification

The following section provides a detailed description of different software components and technologies that are used to design the system under

discussion. This is because each of the software components performs specific tasks that shall be useful when analyzing the code and preparing the documentation. Each software component is made to perform particular operations that are useful in code analysis and documentation preparation and the use of technologies is meant to ensure a smooth integration process.

3.2.1 Python 3.8

The choice of programming language for this project has been Python due to its ease-of-use, flexibility and the availability of community support for artificial intelligence and software analysis utilities. Code parsing, feature extraction, creation of graphs, interfacing with LLMs and the automation of documentation creation is done using Python. The large number of libraries that Python offers, including the NetworkX library for working with graphs and many utility libraries for working with files, helps in implementing difficult functionalities efficiently.

3.2.2 Visual Studio Code

Visual Studio Code (VS Code) has been picked as the primary Integrated Development Environment (IDE) that will be used during development, management and system maintenance. The software offers a very efficient, powerful platform with full Python support and a multitude of development capabilities. These features include syntax highlighting, code suggestions, debugging tools, command prompt integration and many other features that aid in an organized and efficient software development process.

The editor provides a broad set of features including Python functionality, Git functionality and formatting utilities that make coding more efficient. The inbuilt versioning capabilities of the software enable the use of Git projects. In addition, the cross-platform interface provided by VS Code makes it possible to use it across various operating system platforms in developing, debugging and maintaining the proposed system.

3.2.3 Tree-sitter

Tree-sitter is used as an important parser for parsing the code and creating its AST. Tree-sitter offers fast and incremental parsing abilities, making it easy to analyze the code efficiently. Tree-sitter is used to extract many code elements such as file, module, class, function, import and dependency based on the particular programming language. Moreover, Tree-sitter can handle several programming languages, which include Python, Java, C, C++, JavaScript, TypeScript, Go, Rust, Ruby, PHP and Kotlin.

The code tree created via Tree-sitter is highly extensive and provides syntactical as well as semantical elements. Hence, this is beneficial in detecting functions together with arguments and return types with more accuracy. Therefore, the code tree serves as a foundation for further processing of the code by the remaining modules, such as graph generation and feature extraction, more efficiently.

3.2.4 Large Language Model(LLM)

LLM, is utilized to articulate representations of codes into a comprehensive, human-readable format of documentation that interprets and ensures the understanding of the code units correctly. The output is human readable and includes project summaries, architectural summaries and provides a summary for each function.

The LLM then uses its advanced linguistic comprehension to understand programming concepts and code relationships. It has the power to produce human-readable narratives that are brief in length and dependent on the context yet resembling in appearance to human-generated documentation. This structural and relational dual information guarantees correct, but more meaningful and understandable outputs. This helps to reduce human efforts in writing documents without losing the inconsistency and quality of documents produced.

3.2.5 Git

Git is used as the version control system for analyzing the structure of the repository and obtaining historical information regarding the code. This allows the system to work with entire projects by cloning repositories from remote sites, which eliminates manual uploading of separate files. Information at the commit level is also available, for example, for the production of documentation, such as a list of the latest changes.

This would enable the system to be able to identify the different types of changes through commits, thus being able to identify development patterns and get knowledge on how the code is being changed over time. All this would lead to context being better understood, thereby adding a layer of value to the kind of document that is being generated. Other than source code repositories, the system would also allow local source code directories to ensure flexibility with the various types of input sources.

3.2.6 NetworkX

It is used to create and analyze the graphs representing the relationships, function calls and dependencies in the code. NetworkX helps the system to model the code structure as a directed graph, where the functions are represented as nodes and the interactions are represented as edges. This allows the system to understand important parts of the program code, such as the starting point, recursion and any dependencies. The use of graph-based analysis increases the understanding of how the code operates. Additionally, the system uses graph analysis to perform complex evaluations of the code, enabling a better understanding of its critical components.

3.2.7 Markdown

Markdown is used to generate structured and readable documentation files like readme files, to name a few module-level documentation. It makes use of simple formatting tools when producing documentation. Therefore, the resulting files are consistent and easy to read, as Markdown allows structuring

of information in a clear way. They can be shared across different platforms without using special software (accessibility and usability are guaranteed).

In this chapter, we have discussed in detail the requirements related to the hardware, software and development environment that are needed to build an AI-Powered Code Document Generator. These configurations and tools play an important role in ensuring that the process of analyzing code is effective and efficient. With the execution environment described, the next chapter on system analysis discusses the system requirements, functionality and feasibility of the solution.

CHAPTER 4
SYSTEM ANALYSIS

System analysis is the process of structurally examining the proposed AI-Powered Code Document Generator system. The main purpose of the analysis is to comprehend the system's functionality, processing flow and interaction in the creation of accurate and updated documentation from software source code automatically.

The analysis includes an analysis based on how the user uses the whole system, such as a source code directory or a version control system can be provided as input. This includes parsing of the source code, feature extraction and then finally document generation with the help of a LLM. On this case, it is evident that the analysis leads to the production of documents in an efficient way and can be easily maintained and scaled to usable software development purposes. It also ensures an appropriate design of the system that will specify functional and non-functional requirements efficiently and will ensure that the proposed system meets all the user requirements while maintaining accuracy and performance.

4.1 Requirements Analysis

The requirements analysis gives a broad functional and user requirements for documentation using a coding system. It is a requirement that this system will be capable of analyzing the software source code and producing documentation that is easily understood by human readers. All the requirements are specified in the most clear way to be precise, measurable and attainable by the system. The main requirements are that the application should be able to parse source code files, extract structural components of

the program, integrate the language model and provide a user interface to display the documentation. Also, the system should be able to support many programming languages and optimally process different project structure types. These requirements are mandatory to design a reliable, efficient and intuitive system which can generate a large-scale documentation of source code without causing a failure of user expectations and maintaining a level of accuracy and performance.

4.2 Existing System

As of now, software documentation management systems are either based on manual documentation processes or use simple automated documentation systems. In the manual process, programmers create documentation by writing comments, README files and design documents manually. While this approach allows for customization, it requires a significant amount of time and heavily depends on the individual programmer's effort and expertise. Automated documentation systems are also employed to reduce the time required for writing software documentation. These systems generally generate documentation automatically, either using code comments or predefined templates. However, most of these automated systems rely on superficial methods and lack the capability to understand the deeper structural and semantic aspects of the code, limiting their effectiveness in producing thorough and accurate documentation.

4.3 Limitations of Existing System

Though there is access to documentation technology, the existing systems have the following drawbacks:

- Documentation is usually left up to developers to manually update, especially during fast development cycles. Such situations can cause confusion for new developers and hinder maintenance efforts.
- The tools themselves may not fully understand the logic, data flow

or functionality of the code, limiting the generation of insightful and meaningful documentation.

- Some documentation tools struggle to capture the relationships and connections between classes, modules and functions, making it difficult to map out the overall program structure.
- Many current documentation methods support only a single programming language, reducing flexibility and applicability in multi-language projects. In such cases, documentation may become fragmented or inconsistent.
- Automatically generated documentation often lacks context about the code, leaving the purpose, function and interactions of components unclear, which diminishes its usefulness as a reference for developers.

4.4 Proposed System

The proposed system is based on an AI-Powered Code Document Generator that is meant to produce accurate, structured and up-to-date documents automatically, based on the software source code. Differently from other systems, this system combines advanced code analysis with the use of a LLM to understand the structure and semantics of the code. The system accepts input in the form of a local source code folder or a version-controlled repository. Next, the system can use the extracted features to generate meaningful documents such as file descriptions, architectural descriptions or project descriptions.

4.5 Advantages of Proposed System

The proposed system offers the following advantages:

- Minimal manual work is required for documentation generation, as the process is fully automated. This allows developers to focus more on coding rather than spending time writing extensive documentation.

- Precise understanding of the program's code structure is achieved through AST-based analysis, enabling accurate extraction of functions, classes and dependencies. The system provides detailed insights into both the structure and functionality of the code.
- Documentation is human-friendly and context-aware, generated with consideration of the purpose and interactions of specific code elements. This helps developers read and understand the system structure more quickly and efficiently.
- Multi-language support increases versatility, especially for teams working in polyglot environments, eliminating the need for separate documentation tools for different programming languages.
- Efficient handling of large and complex code repositories ensures that the system can provide comprehensive documentation across entire codebases, aiding in the maintenance and navigation of large projects.
- Enhanced understandability and maintainability of source code improve collaboration within teams, ensuring that documentation supports long-term project sustainability and effective knowledge sharing.

4.6 Functional Requirements

This section describes the core functionalities that the system must provide to meet user requirements. Functional requirements define the specific operations and behaviors of the system when analyzing source code and generating documentation. They outline what the system should do and serve as a foundation for designing and developing its various modules.

4.6.1 User Interface

This section offers a summary of the user-system interaction layer. Through a user system interface, the user shall specify the location of local copies of source code files or version control repositories for generating the

documentation. This includes specification of the location of local source code files or version-controlled repositories for documentation generation and the interface will ensure easy interaction with no required technical knowledge or skills.

4.6.2 Source Code Input

This section outlines the supported input formats. The system should accept software projects provided as local directories or version-controlled repositories. It must handle various repository formats and support multiple file types within each project. Additionally, the system should validate all inputs to ensure correctness before initiating the analysis process.

4.6.3 Source Code Parsing

This subsection shows the method that is used for internal source code analysis. To ensure correct understanding of hierarchical code, source code files are parsed through AST generation. Correct parsing results are ensured to identify all the relevant components to work on further correctly.

4.6.4 Feature Extraction

The following section is a description of how meaningful features are extracted from the code. The system parses the source code for structural elements like modules, classes, methods and functions, imports or dependencies. The code is parsed to extract structured elements such as modules, classes and functions, along with imports and dependencies.

4.6.5 LLM-Based Documentation Generation

This subsection describes documentation creation processes. For documentation creation, a dedicated LLM is trained to convert the previously extracted code properties into human-readable documentation. To give the most appropriate human-readable documentation, a specialized LLM

considers the structure and the context of the generated output during the documentation creation procedure.

4.6.6 Documentation Output

The following section describes the various document formats produced by this system will hence be described in the following paragraphs. These are a README file, a description of the architecture, a release of the latest changes and a description at the file level. This means that the output shall be given in such a way that it is concise and clear, making it easier to analyze the document produced.

These are the generated files:

- README.md – Provides a detailed description of the project, including its name, programming language(s), total functions and classes, key imports, overall project description, file structure and instructions for running the code.
- architecture_overview.md – Offers an overview of the system architecture, including per-file responsibilities, module dependencies, inter-module interactions and identification of critical or unused functions.
- recent_changes.md – Lists recent commits, including authors and commit messages, summarizing updates and modifications in the project.
- files/*.md – Contains documentation for each source code file, detailing namespaces, classes, functions, member variables, imports, internal and cross-file relationships, entry points and potential issues such as recursion or unused functions.

4.7 Non-Functional Requirements

The non-functional requirements specify quality attributes for the system. They ensure that the systems output is accurate, consistent and

that it functions effectively and remains user-friendly. The non-functional requirements specify the quality attributes for the system to ensure that it is effective, user-friendly and produces the right output and that it emphasizes on scalable, usability and reliability that underpins implementing a robust and practical AI-powered documentation solution.

4.7.1 Scalability

This system must be able to work with both small projects in source code files and large projects in multi-module repositories. The system's efficiency and uniformity are not compromised with the increasing volume of source codes. The modular design enables multiple files to be simultaneously processed for efficiency and to maintain consistency in performance for large-scale applications.

4.7.2 Usability

The system must provide an easy, straightforward interface to allow users either to upload their source code or provide Uniform Resource Locator (URL) of repositories. The interface should be intuitive and self-explanatory, such that the user is able to generate the documentation without the need for any complex configuration, thereby minimizing any learning requirements for new users.

4.7.3 Reliability

Documentation that is generated should be accurate and well structured. It should be able to cope with such issues as inappropriate or unsupported files without breaking the workflow. This requires an efficient error-handling mechanism that prevents the system from crashing and provides a consistent, stable and reliable performance.

4.7.4 Maintainability

The system's architecture should allow for its customization and upgrading

when changes in programming languages, libraries or documentation norms occur. It is done by using a modular architecture, well-structured code and configurable components that will help the developers customize or upgrade the features with minimum effort, guaranteeing the long-term usability of the documentation system.

4.8 Feasibility Study

This section discusses the feasibility of the proposed system from technical, operational and economic perspectives. The system is considered viable because it uses open-source tools and locally hosted language models, eliminating licensing costs and ensuring practical, cost-effective implementation.

4.8.1 Technical Feasibility

This subsection evaluates the technical feasibility of implementing the proposed system. It relies on widely used and mature technologies, including Python, AST-based parsers and LLMs, all supported by active and robust communities. The system's moderate computational requirements allow it to be developed and executed efficiently in standard computing environments.

4.8.2 Operational Feasibility

This subsection analyzes the usability of the proposed system. It requires minimal user input and little technical knowledge, as users only need to specify the location of the source code file, while the system handles all processing automatically.

4.8.3 Economical Feasibility

This subsection analyzes cost-related aspects. The system uses open-source software and libraries, avoiding any licensing fees and operates on standard hardware, eliminating additional infrastructure costs. This makes it affordable,

cost-effective and accessible for both individual users and institutions.

The analysis of the proposed AI-Powered Code Documentation Generator was carried out by thoroughly examining the system requirements, identifying the limitations of existing documentation solutions and evaluating the functional features of the proposed system. This analysis helps in understanding how the system is expected to behave under different scenarios and ensures that all user requirements are adequately addressed. Furthermore, it highlights the gaps in current approaches and justifies the need for an intelligent, automated solution. Overall, this comprehensive analysis provides clear insight into the system's behavior and establishes a strong foundation for the system design presented in the following chapter.

CHAPTER 5
SYSTEM DESIGN

The design of the AI-Powered Code Documentation Generator is elaborated on in this chapter, emphasizing the organization and structure of system components and the travel of the source code through the pipeline process stages of language identification, feature extraction, AST analysis and the LLM engine until accurate and structured documentation is eventually generated. A modular design approach is used, where every component has an individual purpose in the entire system. This chapter also discusses how the modules interact with each other and how the data flow from the input (source code) to the output (documentation).

5.1 System Architecture

As depicted in Figure 5.1, the architecture of the proposed system represents an AI-driven pipeline for automated code documentation. The process begins when the user provides source code, either by uploading a file or supplying a repository URL. Initially, the input is analyzed by the Language Detection Module to determine the programming language, ensuring that subsequent parsing is accurate. Following this, the system conducts two parallel processes: the Text Feature Extraction Module captures surface-level code features, while the AST Analysis Module generates a structured representation of the code's components.

The AST output is then sent to the Graph Building Module, where a graph is constructed with nodes representing functions or classes and edges representing relationships such as function calls and dependencies. The outputs from the Text Analysis Module, AST Module and Graph Building Module

are then combined in the Feature Fusion Module to produce a comprehensive representation that integrates the textual, structural and relational aspects of the code.

This integrated representation is then fed into the LLM Module, which produces human-readable documentation. The Documentation Module organizes this content into various document formats and delivers the finalized documentation to the user.

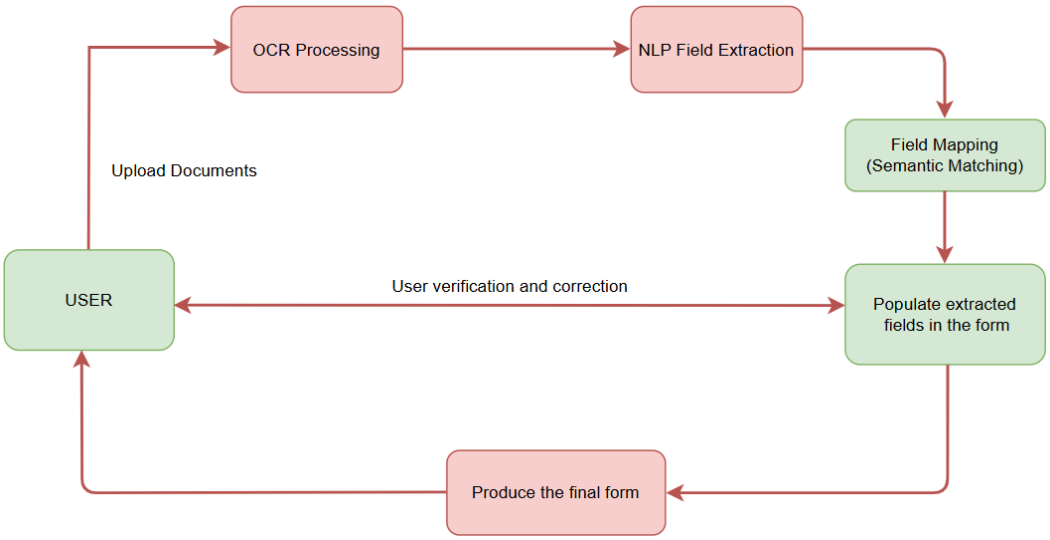


Fig. 5.1: Architecture Diagram of the AI-Powered Code Documentation Generator.

5.2 Data Flow Diagram (DFD)

Figures 5.2 and 5.3 illustrate the data flow diagrams of the proposed system, showing the complete movement of data throughout the automated code documentation generation process.

5.2.1 DFD Level 0

Figure 5.2 demonstrates the Level 0 Data Flow Diagram (DFD) of the AI-Powered Code Documentation Generator. The user provides source code to the system, which uses AST analysis to extract essential details such as functions,

classes, imports and other relationships. Based on this extracted information, the AI generates structured documentation and delivers it to the user.

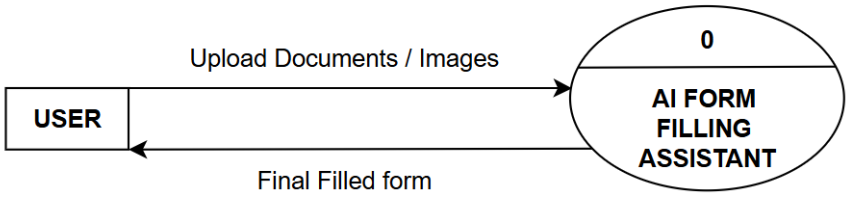


Fig. 5.2: Data Flow Diagram Level 0.

5.2.2 DFD Level 1

Figure 5.3 presents the internal processing of the AI-Powered Code Document Generator as a Level 1 DFD. The process begins when the user uploads the source code and the Language Detection Module identifies the programming language based on the file extension. Once the language is detected, two parallel streams occur: the Text Feature Extraction Module extracts textual information and code metrics, generating text data, while the AST Extraction Module parses the code to create its abstract syntax tree, representing its structural organization. The AST is then passed to the Graph Feature Extraction Module to produce graph-based features capturing relationships among functions, classes and method calls. All features are merged in the Feature Fusion Module, which is subsequently processed by the LLM Engine (Ollama) to generate a natural language description of the source code. Finally, the Document Generation Module formats this output into structured documentation for the user.

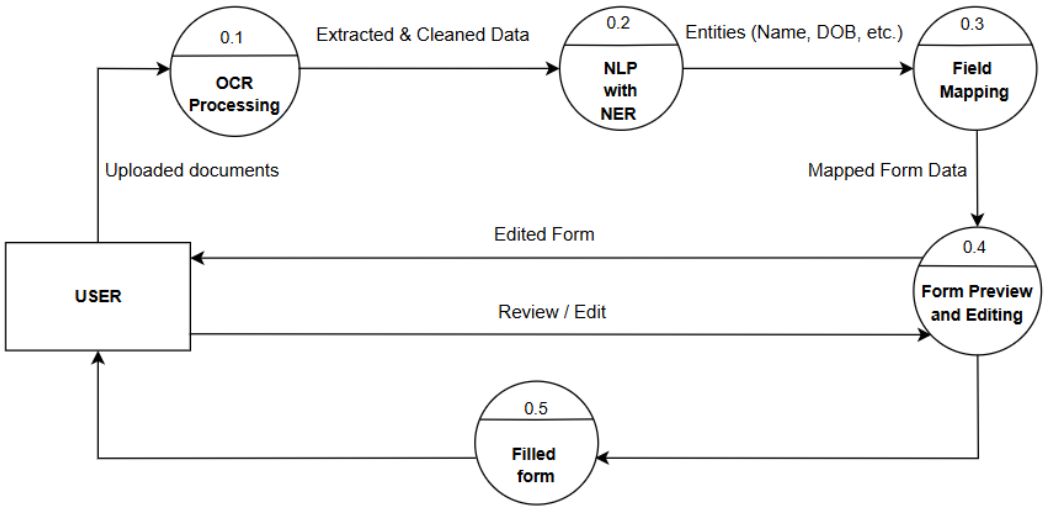


Fig. 5.3: Data Flow Diagram Level 1.

5.3 List of Modules

The AI-Powered Code Documentation Generator is composed of the following modules:

- User Interface Module
- Language Detection Module
- Text Feature Extraction Module
- AST Analysis Module
- Graph Building Module
- Feature Fusion Module
- LLM Engine Module
- Documentation Output Module

5.3.1 User Interface Module

This module serves as the main interface between the user and the system. It allows users to provide source code files, project directories or repository URLs and access both the generated documentation and the results of intermediate analysis. The interface ensures smooth navigation through each step of the process, making the analysis workflow seamless for the user.

5.3.2 Language Detection Module

The Language Detection Module identifies the programming language of the uploaded source code by analyzing file extensions and syntax. This detected language guides the system in selecting the appropriate parsing strategies and analysis methods. Accurate identification ensures correct AST generation and feature extraction, allowing the system to efficiently process multi-language projects and improving overall flexibility and adaptability.

5.3.3 Text Feature Extraction Module

This module extracts text-based features from the source code, calculating metrics such as the number of lines, tokens, comments, identifiers and keywords. It also identifies import statements, revealing external libraries and dependencies used in the project. These features are critical for generating project-level documentation, including README files and technology stack summaries, providing a comprehensive overview of the codebase. The module operates independently of language-specific syntax, making it fast and lightweight.

5.3.4 AST Analysis Module

This module analyzes the structure of the source code using an AST. It provides detailed insights into the internal structure of the code by extracting classes, functions, parameters, return types, variables, imports and function calls. As a core module, it ensures accurate and meaningful documentation

based on the code itself rather than assumptions. It also supports language-aware analysis to enhance extraction accuracy and efficiency.

5.3.5 Graph Building Module

This module constructs a semantic graph from the AST. Nodes represent code elements such as files, classes and functions, while edges indicate relationships like function calls, imports and inheritance. The graph provides a clear visualization of interactions among code components, illustrating data flow and module dependencies. It also supports deeper analysis, including the identification of recursion, unused functions and critical components.

5.3.6 Feature Fusion Module

This module integrates textual features, AST-based structural features and graph-based relational features. Textual features provide specific information, AST features detail the functionality of each component and graph features illustrate the relationships between components. By combining all these features, the system can generate more accurate and comprehensive documentation, ensuring that no information is lost during processing.

5.3.7 LLM Engine Module

This module uses a LLM to create human-readable documentation. The input to the module consists of feature summaries created from the AST, text and graph analysis. The model uses the structural and contextual information to create accurate documentation about the functionality of the code and the architecture of the system. This ensures the readability, accuracy and appropriateness of the created documentation. The level of detail in the documentation depends on the complexity of the input code.

5.3.8 Documentation Output Module

This module handles structuring the generated documentation in an organized layout, covering areas such as project overview, architecture, API

documentation and module-level details. It ensures consistency in presentation, enhancing readability and usability. The documentation can be exported to formats like Markdown, making integration with existing development workflows easier and supporting efficient sharing and management.

In this chapter, the overall system design of the AI-Powered Code Documentation Generator is discussed, including the system architecture, the functionality of its various modules and the complete data flow. This design enables each module to process the source code independently while ensuring that the analysis and documentation generation proceed systematically and cohesively. With the system's design and architecture established, the next chapter will cover the implementation aspects, detailing the development of each module and how they integrate to achieve the intended functionality.

CHAPTER 6
SYSTEM IMPLEMENTATION

The system implementation phase is aimed at realizing the AI-Powered Code Documentation Generator design. At this stage, full development occurs, including the implementation of all system modules, such as the code parser, structural feature extraction algorithm, semantic understanding algorithm and automatic documentation generator. Each module is developed and implemented independently, allowing developers to make changes or enhancements without affecting other modules. This approach ensures high scalability, as modifications and additions can be implemented easily while maintaining the integrity and efficiency of the system.

6.1 Setup Development environment

The importance of having an efficient development environment cannot be ignored when implementing and executing the system. In the context of the current system, the VS Code editor is chosen as the development environment for its lightweight structure and easy integration with different tools. The backend is designed with the Python 3.8 language because of the numerous benefits that come with the use of this language when processing data and implementing machine learning algorithms. The HTML (HyperText Markup Language), CSS (Cascading Style Sheets) and JavaScript languages are used to implement the frontend of the system to make the interaction experience of the user more engaging. The backend server has been implemented using the Flask web framework, which helps in creating RESTful APIs for the smooth communication between the frontend and backend modules. To support the implementation of various functionalities, the following Python libraries are

installed and configured:

- Flask – This tool is used in constructing the backend server as well as the REST (Representational State Transfer) API endpoints. It manages all requests made by users, like uploading files. This too starts the code analysis and sends back the documentation that is created.
- Tree-sitter – This tool is used for parsing source code and converting it into ASTs, enabling accurate syntactic analysis for multiple programming languages. It divides the source code into structural elements such as functions, classes and other components.
- Tree-sitter-languages – This library provides built-in language support that extends Tree-sitter's capabilities, allowing the system to support multiple programming languages such as Python, Java, JavaScript, C, C++, Go, Rust and Kotlin without the need for separate parser configurations for each language.
- Markdown – A Markdown library provides tools to parse, render and manipulate Markdown content programmatically. Such libraries support creating dynamic documentation, reports or web content from plain text. They simplify integrating Markdown-based documentation into automated systems and applications.
- Networkx – This tool is used to construct and analyze graph structures derived from the parsed code helping to achieve a deeper contextual understanding of the code.

6.2 User Interface

The User Interface is built as a single-page application with HTML, CSS and JavaScript, enabling users to upload source code files, folders or even input a public Git repository URL. The User Interface supports three modes: Single File, Folder and Mixed.

The frontend sends a request to the /analyze endpoint to communicate with the Flask-based backend. Once the request is received, the backend will process the input by calling the core function analyze_path(). This function will determine whether the input is a file, folder or Git URL and then route it to the appropriate analysis pipeline. If it is a Git URL, the repository will be cloned to a temporary directory and then cleaned up. If it is a local input, the function will directly call either the single-file analysis pipeline or the project-level analysis pipeline based on whether the path is pointing to a file or a directory.

The generated documentation will be displayed in the browser as formatted HTML using the marked.js library. Users will also be able to download the complete generated Markdown files as a ZIP package using the JSZip library.

6.3 Language Detection

The next step is to recognize the programming language of each uploaded source file based on its file extension. This is done by utilizing an extension-to-language mapping defined in language_config.py, which allows the system to support twelve programming languages including Python, Java, JavaScript, TypeScript, C, C++, Go, Rust, PHP, Ruby, Kotlin and C#.

During processing, the file extension is extracted from the file path, which is converted to lowercase for standardization purposes, as different operating systems have different capitalization standards. After that, the programming language is detected based on the predefined file extension mapping.

In order to ensure robustness, the unsupported file types will be skipped during the traversal of the folders. Also, empty files will be removed prior to the start of the analysis process. Moreover, the Git metadata directories such as .git will be skipped during the traversal of the folders using the normalized paths where backslashes will be replaced with forward slashes in order to support both the Windows and Linux platforms, thus preventing the Git source code from being treated as source code.

6.4 Text Feature Extraction

In this stage, lightweight textual features are obtained from individual source files without reference to the AST, aiming to identify basic structural properties of the code. The features are calculated through text-based analysis in text_features.py without any reference to language parsing or grammar.

The function reads the source file in a safe manner with UTF (Unicode Transformation Format)-8 encoding and ignores decoding errors for robustness across different file types and legacy codebases. In case the file is invalid or cannot be read, an empty dictionary is returned.

The features that are extracted are the raw counts, which include the total number of lines, the total number of characters, the number of import statements, the number of class definitions and the number of function definitions. These features offer an overview of the source file.

These metrics are attached to every function record as a metrics field. They help in the formation of the unified fused representation in the downstream modules.

6.5 AST Analysis

In this stage, structural information is obtained through structural feature extraction, where each source file is parsed to an AST with the help of the tree-sitter framework, facilitating precise analysis for all fourteen programming languages.

The entry point is tree_sitter_adapter.py where the programming language is resolved based on the file extension, the source file is read in a safe manner with UTF-8 encoding and the parse tree is generated along with a source representation at the byte level to accurately extract the text within nodes. Node type names such as function definitions, class declarations and call expressions for different languages are read from language_config.py to ensure that the logic itself doesn't have any explicit language checks.

The AST is processed in ast_graph.py with a two-pass strategy. The first

pass traverses the entire AST tree structure to collect all the function names, class names and import symbols in the known_names set. The known_names set will be used in the second pass of the AST traversal process to differentiate calls to functions defined within the project codebase from calls to external library code. The known_names set is supplemented with symbols obtained from all the files in the codebase in the global pre-pass in analyzer.py.

The second pass carries out full per-function extraction. For a function, the system extracts the function's name, class membership, parameters, return type, internal function calls, external function calls, local variables and string literals. For function names, a four-strategy cascade is used to accommodate the different structural characteristics of programming languages. Most languages rely on a direct field lookup for function names, while the languages C and C++ rely on a declarator chain walk and a shallow identifier scan and a deep recursive scan serve as a fallback.

Although function calls are identified during this process, the caller-callee relation is not stored as a string. Rather, every identified internal call is directly added to the NetworkX graph by invoking graph.add_edge(). This graph is the sole source of all call relation data. The extracted function-level metadata is stored in the metadata dictionary and passed to the next stage, Graph Analysis.

6.6 Graph Analysis

Within this stage, the semantic graph created during the AST traversal process is examined to determine higher-level structural information about the codebase. The graph is built using the NetworkX library, in which the nodes of the graph represent the functions and the edges represent the caller-callee relations. The graph is the single source of truth for all call relationship data, which means the data about the relations of the functions is not obtained from intermediate lists of strings or the AST.

The call relationships in the graph are obtained by using successor queries on the graph, where the outgoing edges on a function node directly provide

the list of functions it calls. This method ensures consistency throughout the system and avoids the need for string parsing and redundant storage.

In addition to storing the relationships between the entities, the graph is used to find four different structural characteristics in the code. The entry points in the code are the set of functions that have no incoming edges but have outgoing edges; in other words, these are the functions that are never called by any other function in the code but start the calls themselves. The dead functions in the code are the set of functions that have neither incoming nor outgoing edges; in other words, these are the functions that are never called in the code and never start any calls either. A cross-file check is done before determining the dead or entry point functions in the code.

Direct recursion is identified through self-loop edges in the graph, where a function has an edge pointing back to itself. Mutual and chain recursion are detected using cycle detection algorithms, which identify groups of functions forming circular call chains of length greater than one. Critical functions are determined by ranking nodes based on their in-degree, where functions with a higher number of incoming edges are considered the most depended-upon in the codebase.

These features of the derived graphs are included in each function's record and passed on to the Feature Fusion and Documentation Generation stages, which enrich the LLM prompt with architectural information that cannot be automatically inferred from the AST features.

6.7 Feature Fusion

At this stage, features extracted by the Text Feature Extraction, AST Analysis and Graph Analysis modules are integrated into a single structure for each function. This integration encompasses the structural, relational and contextual aspects of the code in a single structure, which is used by all the modules.

The fusion is achieved in analyzer.py through the _build_record() function, which constructs one record per function by combining three different sources

of information. The information from the AST is used for the structural facts about each function, including its name, its membership in a class, parameters, return type, external calls, local variables, string literals and imports. The information from the graph is used for the relational facts, including the list of internal calls and the formatted caller-callee relationship strings, both of which are obtained directly from graph successor queries. The text features include the contextual metrics, including the raw counts used by the LLM..

The resulting fused records are serialized into semantic_records.json, which is used as the persistent interface between the analysis pipeline and the module for generating the documentation.

6.8 Documentation Generation

The Documentation Generation Module uses the open-source LLM DeepSeek, which is hosted locally via Ollama, to create structured Markdown documentation from the integrated code records. The model is accessed via the Ollama HTTP API with a low temperature of 0.1 to ensure consistent and factually reliable outputs. The stream: false parameter ensures that the complete response is returned as a single JSON (JavaScript Object Notation) payload, simplifying downstream processing.

For each function, a structured fact block is created from the fused record and passed into the LLM as a prompt. This fact block combines all three sources of features into one line of context.

All documents are generated in parallel by a ThreadPoolExecutor with up to six concurrent workers, greatly improving the total generation time for large projects. If a document does not generate correctly, it will have a stub so that the pipeline always produces a complete output even in the presence of individual failures

The module produces four types of Markdown documentation artifacts per run of the analysis:

- README.md for project overview, description, tech stack, project structure, etc.

- architecture_overview.md for component responsibilities, data flow between files, module dependencies, etc.
- File-wise .md files for file-wise information like import information, member variables, function intents, parameters, return types, function call relationships, entry points, dead code, recursion and cycles.
- recent_changes.md for displaying Git commit history with author information and message for every commit, if input is provided as a Git repository URL.

The implementation phase has been successfully completed to transform the proposed design into a fully functional form of the AI-Powered Code Document Generator. All the modules, ranging from language detection and AST parsing to graph analysis, feature fusion and LLM-based generation, have been implemented and integrated to function cohesively in a complete system. Since the system has been fully operational for a range of programming languages and input types, the discussion will move on to the evaluation stage. The next chapter will cover the results achieved by running the system on a range of projects and a discussion on the quality, accuracy, and completeness of the generated documentation.

CHAPTER 7
RESULTS AND DISCUSSION

This chapter presents and analyzes the results obtained from the proposed system, the AI-Powered Code Documentation Generator. The system was evaluated using source code files provided through two input methods: direct file uploads via the web interface and repository-based inputs through public Git URLs. This dual-mode testing ensured that the system was assessed under both isolated and real-world development scenarios.

The results demonstrate that the system is capable of effectively processing multi-language codebases and generating well-structured, coherent and contextually relevant documentation. By integrating techniques such as syntactic analysis, structural feature extraction and language model-based generation, the system successfully transforms raw source code into meaningful documentation. Overall, the observed outputs highlight the system's reliability, scalability and practical applicability in automating documentation tasks across diverse programming environments..

7.1 Results

The system was tested in different input modes to evaluate its functionality and output quality. The following figures illustrate the working of the system and the documentation generated at various stages.

The screenshot in Figure 7.1 displays the homepage of the AI-Powered Code Documentation Generator when in file upload mode. There are two modes in which a user can input the source code. These modes are Upload Files and Repository URL.

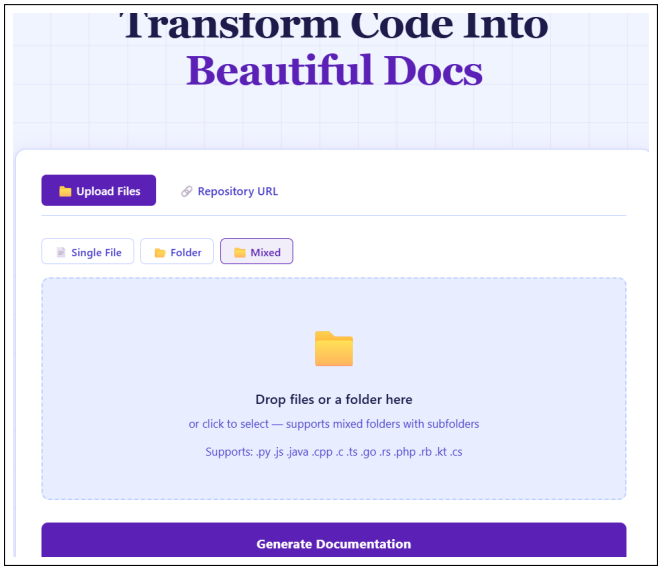


Fig. 7.1: Complete User Interface of the System.

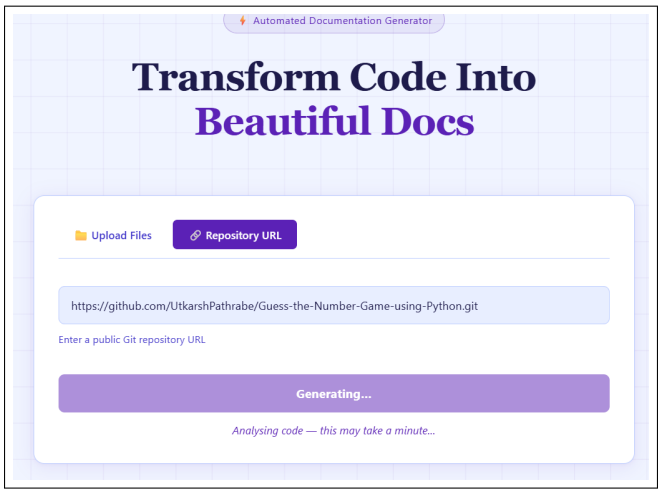


Fig. 7.2: Home Page – Repository URL Input Mode.

Figure 7.2 shows the User Interface (UI) for the homepage with the Repository URL mode selected. The user can provide the public git repository URL in the text box. The system then downloads the repository in a temporary directory and processes all the source files from the repository without the need to upload the files.

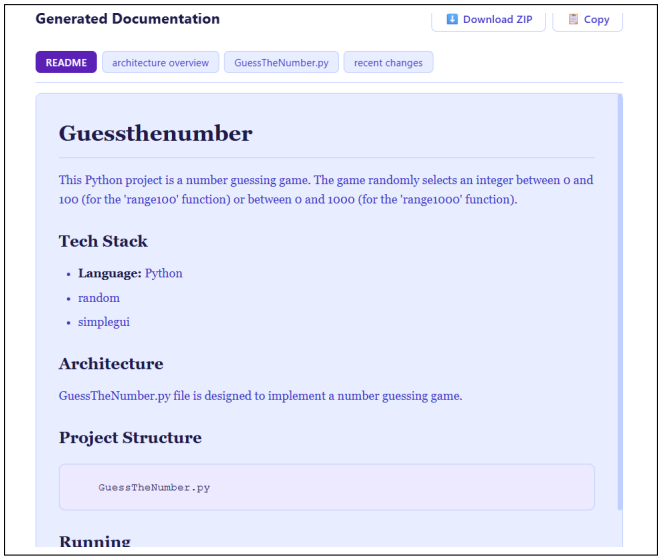


Fig. 7.3: README.md – Project Overview Output.

The system shows the README tab of the generated documentation output in Figure 7.3. It analyzes the repository and provides a project-level summary, including project description, technology stack, architecture summary, run instructions, and project structure.

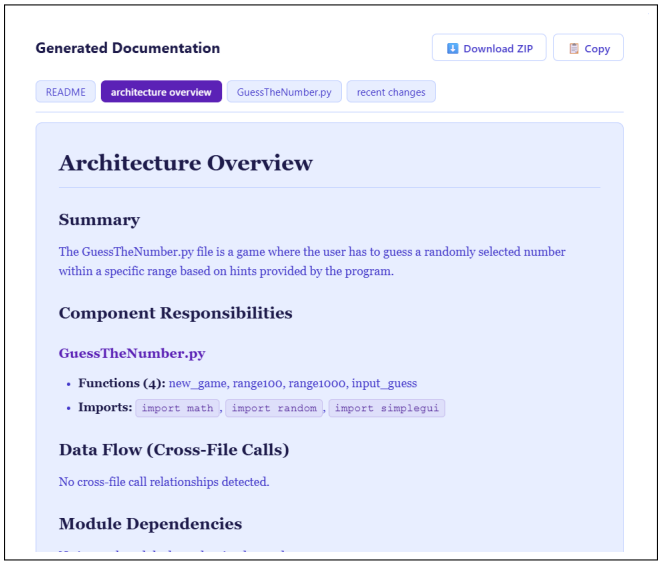


Fig. 7.4: architecture_overview.md – Architectural Description Output.

Figure 7.4 shows the system displaying the Architecture Overview tab of the automatically generated documentation. The system generates a structured architectural description consisting of file-wise summary

information, component responsibilities such as classes, functions, imports and fields, call relationships between files representing data flow and module dependencies from both import statements and function interactions.

Figure 7.5 shows the file-wise documentation generated for the application in Markdown format. Each file has its documentation with information like function intent, parameters, return values, function calls, local variables and internal call relationships, along with graph-based information like entry points, unused functions, dependencies and recursion information.

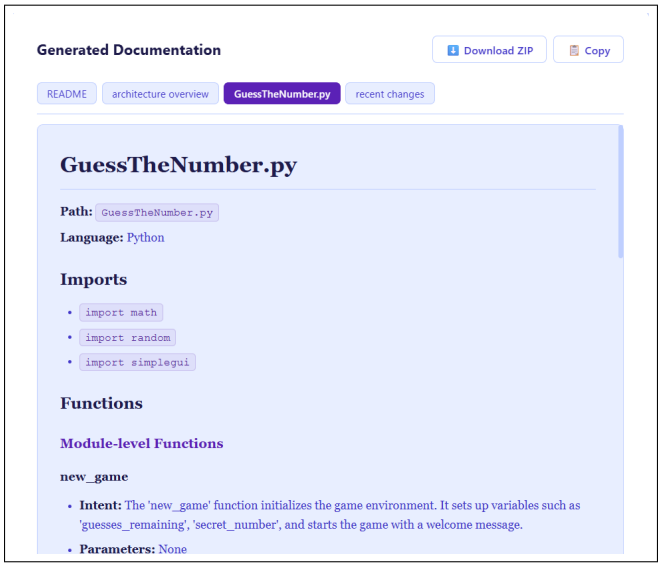


Fig. 7.5: File-wise .md Output.

Figure 7.6 shows the Recent Changes tab of the automatically generated documentation. The system has automatically collected the commit history from the Git repository and created a structured document named recent_changes.md. This feature helps users easily track project updates and understand the evolution of the codebase over time.

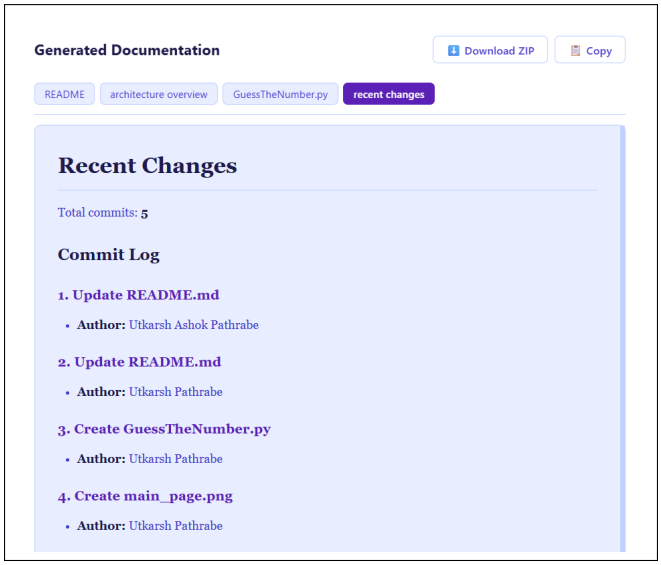


Fig. 7.6: recent_changes.md – Git Commit History Output.

7.2 Discussion

The results show that the proposed system can efficiently produce overall documentation on different levels, including projects, architecture and functions. The accuracy in the extraction of structural information is guaranteed by AST-based analysis, while graph-based analysis allows for precise identification of relationships between functions. The inclusion of text-based metrics improves the results produced by the system, enhancing their overall relevance and readability.

The system works effectively with different programming languages, thus demonstrating the flexibility of the tool. This is further evidenced by the repository URL mode, which is beneficial since the entire projects can be analyzed without the need for file handling. Furthermore, the inclusion of the history of commits is advantageous since it provides additional information compared to other documentation tools.

However, the system is not without certain limitations. The quality of the code fed into the system will affect the quality of the documentation that is created. Also, the dynamic behavior of the code is not fully taken into consideration in the system. Nevertheless, the system is useful in reducing the effort required in understanding the code.

CHAPTER 8
CONCLUSION

The proposed AI-Powered Code Document Generator would thus offer an intelligent and automated solution for the generation of accurate, structured, and context-aware documentation from source code. This would thus help in the identification of significant software components like functions, classes, imports, call relationships, entry points, and recursive patterns, among others. In this regard, the proposed solution would leverage Tree-sitter for parsing code across fourteen programming languages and thus generate accurate documentation like the README file, architecture, description of files, recent changes of the git repository, among others.

The system relies on a three-stage feature extraction process to guarantee the LLM access to rich and non-redundant information for every function it documents. Features extracted from the AST offer structural information such as the function's name, class membership, parameters, return type, and calls. Features extracted from the text offer context-based information on the file's complexity and documentation. Features extracted from the graph offer architectural information on the function and the program based on the NetworkX library's directed call graph. By combining all the features in a single fact block, the system offers documentation that is functionally correct, contextually scaled and architecturally aware.

This modular and language-independent design ensures that each module is independent in its functioning through well-defined interfaces. It is possible to upgrade or extend any module in the pipeline without affecting the rest of the pipeline. Cross-file awareness ensures that functions used across module boundaries are correctly identified as active rather than unused, thereby

improving the reliability of the architectural analysis. The parallel document generation using the ThreadPoolExecutor ensures the pipeline scales well with projects of different sizes.

This tool is highly beneficial for software development teams working with repositories that lack adequate documentation. It boosts productivity by automating what is usually a manual and time-consuming process. Additionally, it supports long-term code maintainability by allowing developers to understand the code's structure through graph-based analysis.

Overall, the project highlights the potential of combining artificial intelligence with static code analysis to create a practical tool that simplifies software documentation. Future improvements could include extending support to more programming languages, integrating with version control systems to enable real-time documentation updates and employing more advanced language models for richer, more detailed explanations, ultimately developing the system into a comprehensive, production-ready documentation platform.

Despite the current system addressing many key challenges in automated documentation, there is room for improvement by enhancing system capabilities, supporting additional programming languages and adding new features to make the system more intelligent and adaptive. These opportunities pave the way for future developments that could increase the system's effectiveness and applicability in real-world software development scenarios which discussed in next chapter.

CHAPTER 9
FUTURE ENHANCEMENT

The AI-Powered Code Document Generator sets a robust foundation for automated documentation, demonstrating how static code analysis and large language models can work together to produce accurate, structured and context-rich documentation for multiple programming languages. This chapter explores several opportunities for enhancing and refining the system to advance it toward a more comprehensive, production-ready platform.

9.1 Git Integration

One of the way to improve the system would be to integrate it more closely with Git-based platforms like GitHub and GitLab. Currently, documentation is generated when a user provides a repository URL or manually uploads files. Enhanced Git integration could allow the documentation to update automatically with each commit.

9.2 Multi-Language Expansion

Although the system currently supports fourteen programming languages, future upgrades could add support for languages like Swift, Scala, Dart and Haskell. Expanding language compatibility would make the system applicable to a wider range of development projects beyond its current scope.

9.3 Interactive Q&A

A highly valuable future feature could be the addition of an intelligent Question&Answer system (Q&A), allowing developers to interact with the

codebase through a natural language interface. Instead of merely reviewing the automatically generated documentation, developers could directly ask questions about function call chains, recent changes or other specific aspects of the system.

The future improvements proposed in this chapter represent a natural progression of the current system. Git integration would embed documentation generation into the ongoing development workflow. Expanding multi-language support would broaden the system's applicability across diverse programming environments. Adding an interactive Q&A feature would transform the system from a passive documentation generator into an active tool for code understanding. Collectively, these enhancements would result in a highly efficient and practical automated documentation platform.

BIBLIOGRAPHY

[1] J. Song, Z. Zhang, Z. Tang, S. Feng, and Y. Gu, "Improving Code Summarization With Tree Transformer Enhanced by Position-Related Syntax Complement," IEEE Transactions on Artificial Intelligence, vol. 5, no. 9, pp. 4776–478X, Sept. 2024, doi: 10.1109/TAI.2024.3395231.

[2] M. F. Rozi, T. Ban, S. Ozawa, A. Yamada, T. Takahashi, S. Kim, and D. Inoue, "Detecting Malicious JavaScript Using Structure-Based Analysis of Graph Representation," IEEE Access, vol. 11, pp. 102727–1027XX, 2023, doi: 10.1109/ACCESS.2023.3317266.

[3] M. M. Ghahfarokhi, A. Khademian, S. Kianiangolafshani, A. Asadi, H. Jahantigh, and A. Heydarnoori, "Beyond Syntax: Unleashing the Power of Computational Notebooks Code Metrics in Documentation Generation," in Proc. IEEE/ACM Int. Conf. AI Engineering – Software Engineering for AI (CAIN), Lisbon, Portugal, Apr. 2024, pp. 278–280, doi: 10.1145/3644815.3644979.

[4] W. Wang, Y. Zhang, Y. Sui, Y. Wan, Z. Zhao, J. Wu, P. S. Yu, and G. Xu, "Reinforcement-Learning-Guided Source Code Summarization using Hierarchical Attention," IEEE Transactions on Software Engineering, 2020.

[5] S. Yao, Y. Shen, D. Liu, and J. Hu, "CodeDocAgent: Leveraging Large Language Models for Accurate and Contextual Code Documentation," in Proceedings of IEEE, 2025.

[6] Y. Zhu, X. Zhao, X. Li, Y. Zou, H. Yuan, Y. Wang, and B. Xie, "RepoSummary: Feature-Oriented Summarization and Documentation Generation for Code Repositories," in Proc. ACM Conf., 2025.

[7] J. Y. Khan and G. Uddin, "Combining Contexts from Multiple Sources for Documentation-Specific Code Example Generation," in Proc. IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER), 2023, pp. 683–693, doi: 10.1109/SANER56733.2023.00071.

[8] S. R. Thota, S. Arora, and S. Gupta, "AI-Driven Automated Software Documentation Generation for Enhanced Development Productivity," in Proc. Int. Conf. Data Science and Network Security (ICDSNS), 2024.

[9] D. P. Ghale and M. Dabbagh, "Automated Code Comments Generation Using Large Language Models: Empirical Evaluation of T5 and BART," IEEE Access, vol. 13, pp. 141420–1414XX, 2025, doi: 10.1109/ACCESS.2025.3597601.

[10] T. T. Procko and S. Collins, "Automatic Code Documentation with Syntax Trees and GPT: Alleviating Software Development's Most Redundant Task," Embry-Riddle Aeronautical University, Daytona Beach, FL, USA.